



Deep Learning Term Project

Image-Based Classification and Captioning Model for Recycling Used Clothes with Contamination Annotations

오염 요소 기반 폐의류 재활용 분류 및 이미지 캡셔닝 모델 개발



Minji Kim, Kaeun Han
Ewha Womans University

Contents

01 Problem Definition

02 Introduction

03 Data Collection

04 Object Detection : YOLOv8

05 Image Captioning : BLIP

06 Potential and Applications

01 Problem Definition

01 Problem Definition



Significant textile waste is reusable and recyclable, but often discarded.

Garment sorting remains predominantly manual - **limiting efficiency and accuracy.**

Annually, 9200 tons of fiber waste are generated.

01 Problem Definition



If recycling feasibility can be determined based on contamination,

Could this contribute to efficient resource circulation and waste reduction?

02 Introduction

02 Introduction

- Topic : Image-based classification and captioning model for recycling used clothes based on contamination

- Project Overview :

Detect clothing and contamination in images, classify each into reuse, recycle, or dispose categories, and generate automated explanatory captions for each item.

- Key Technologies :

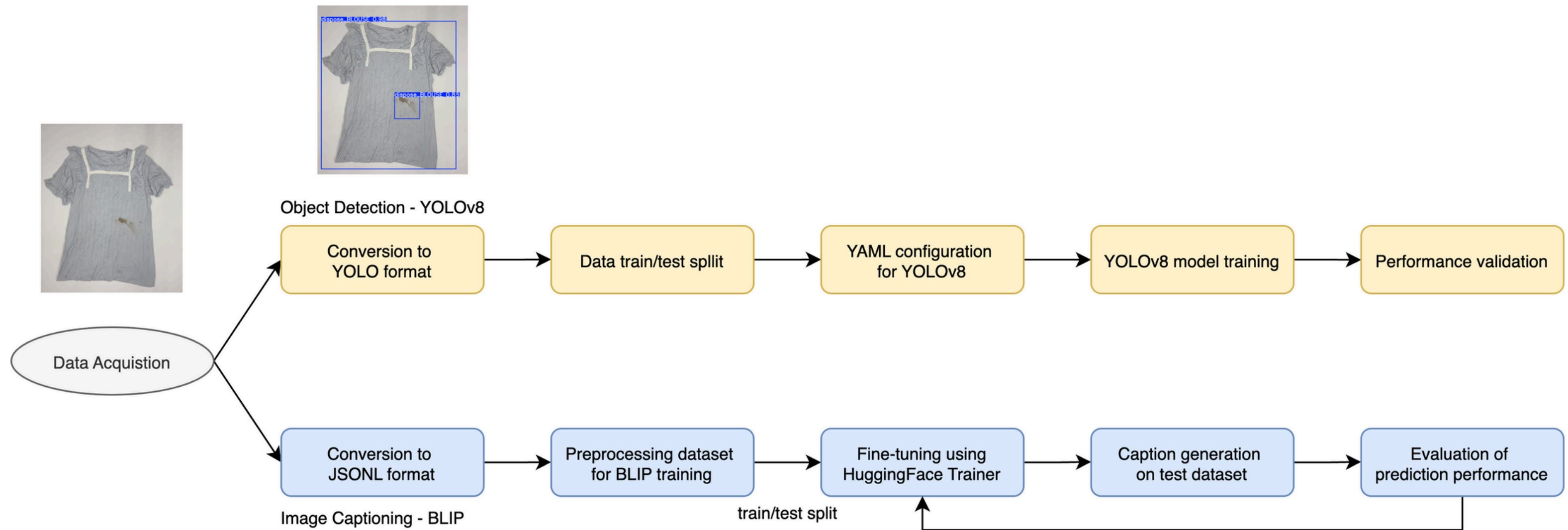
Object Detection : Identify clothing and contamination bounding boxes (YOLOv8)

Classification : Categorize detected items into reuse / recycle / dispose

Image Captioning : Automatically generate item descriptions using visual information (BLIP)

02 Introduction

Project Workflow



예측된 캡션: 흰색과 스트라이프 패턴이 중앙에 여성용 반팔 블라우스
실제 캡션: 우측 중앙에 외부오염이 있는 하늘색 여성용 반팔 드레스

03 Data Collection

3-1 Data Collection

About Dataset

- **Dataset** : Waste clothing recyclability classification and selection dataset
- **Source** : AI Hub (NIA)
- **Dataset Summary** :
 - Raw images : 260,000, Labeled data : 260,000 (paired 1:1)



메타데이터 구조표

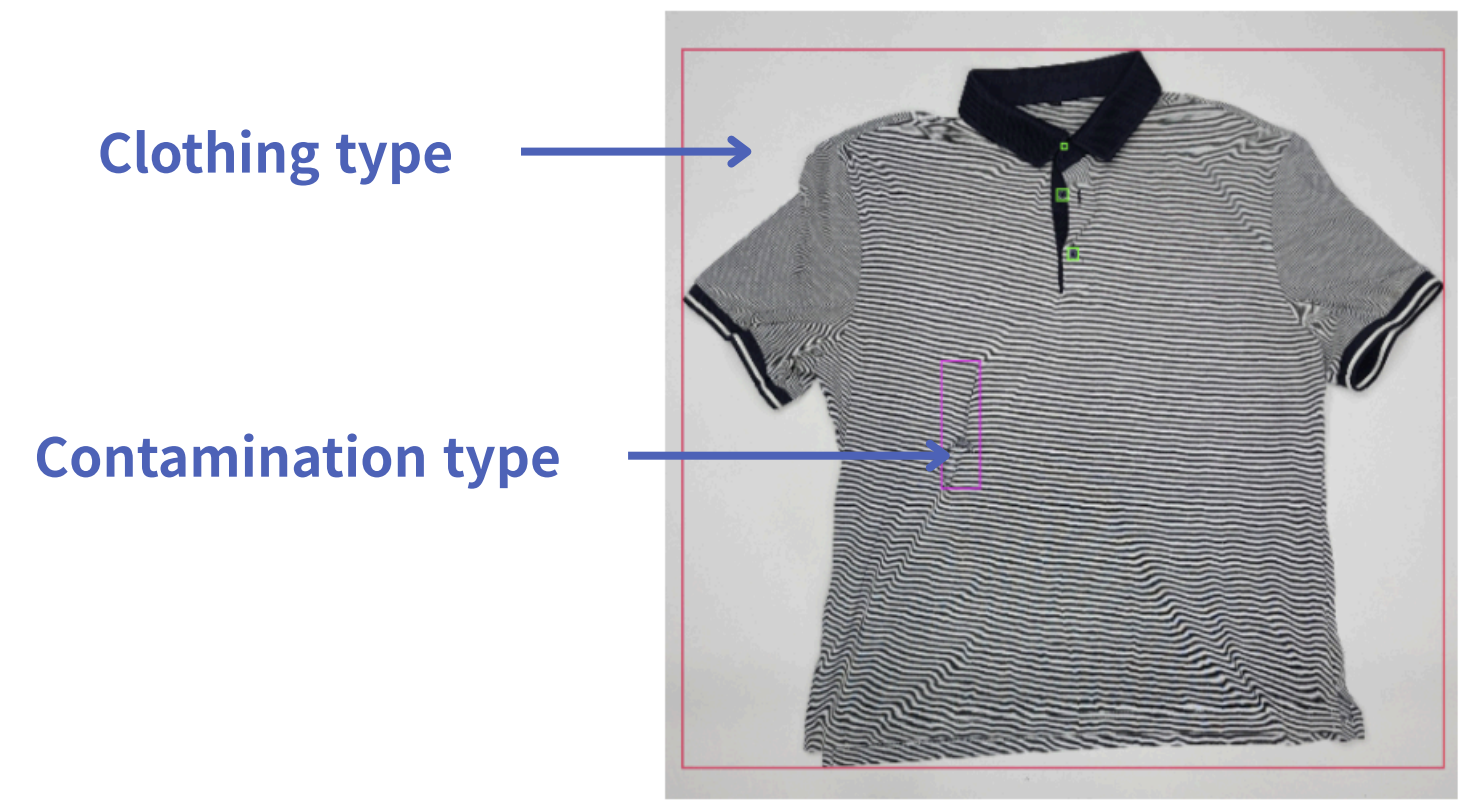
데이터 영역	재난안전환경	데이터 유형	텍스트, 이미지
데이터 형식	JPG	데이터 출처	직접 촬영
라벨링 유형	바운딩박스(이미지), 질의응답(자연어)	라벨링 형식	json
데이터 활용 서비스	폐의류의 분류 및 손상, 오염, 부착물 탐지	데이터 구축년도/ 데이터 구축량	2024년/260,000set

- **Dataset Classification Criteria** :
 - **Contamination type**
→ Classified into reuse / recycle / dispose
 - **Clothing type** → Total of 11 categories

3-1 Data Collection

Data Filtering & Selection Criteria

- **Issue** : Out of 260,000 total images, there are significant variations and class imbalances across clothing categories → For this projet, we focused on **upper-body clothing (Blouse + Shirts)** to improve data quality and model accuracy.
- **Selection Rationale**
 - Balanced distribution of comparative images
 - Representative for future expansion to diverse clothing categories



[재활용 의류 예시]

Example of waste clothing image bounding box labeling

3-2 Data Storing : Challenge

Data Download Attempts

- Attempted to download from AI Hub using CLI
- Attempted direct file download from Google Cloud VM Instance as .zip format

Issues Encountered

- Google Cloud VM (asia-northeast3) is in the Korea region, but AI Hub identified it as a foreign IP.

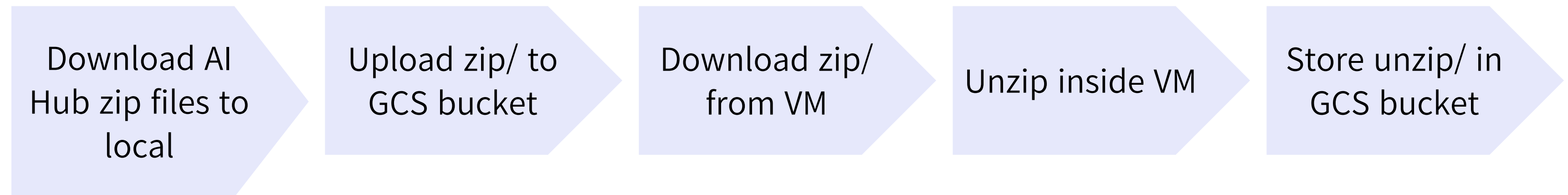
→ Download failure (502 Bad Gateway error)

Resolution Strategy

- Establishment and implementation of alternative strategies.

3-2 Data Storing : Challenge

Solution



Google Cloud CLI-based upload process

Parallel copy of the entire labeling data folder from GCS bucket to local folder

```
gsutil -m cp -r gs://deeplearn-bucket-1/Training/라벨링데이터 ~/Training/ ← download to VM instance
```

Unzip and re-upload to GCS bucket

```
cd ~/Training/라벨링데이터
```

```
for ZIPFILE in *.zip; do
```

```
  FOLDER="${ZIPFILE%.zip}_unzipped"
```

```
  unzip -o "$ZIPFILE" -d "$FOLDER" ← unzip
```

```
  gsutil -m cp -r "$FOLDER" "gs://deeplearn-bucket-1/Training/라벨링데이터/$FOLDER/" ← re-upload to GCS
```

```
done
```

3-2 Data Storing

List of uploaded data

Raw images	dispose	TS_task1_yolo_dispose_BLOUSE.zip	951.77 MB	Labeled data	dispose	TL_task1_yolo_dispose_BLOUSE.zip	301.50 KB
		TS_task1_yolo_dispose_SHIRTS.zip	1.29 GB			TL_task1_yolo_dispose_SHIRTS.zip	363.20 KB
	recycle	TS_task1_yolo_recycle_BLOUSE.zip	2.67 GB		recycle	TL_task1_yolo_recycle_BLOUSE.zip	882.49 KB
		TS_task1_yolo_recycle_SHIRTS.zip	3.45 GB			TL_task1_yolo_recycle_SHIRTS.zip	1.02 MB
	reusable	TS_task1_yolo_reusable_BLOUSE.zip	4.50 GB		reusable	TL_task1_yolo_reusable_BLOUSE.zip	1.19 MB
		TS_task1_yolo_reusable_SHIRTS.zip	9.49 GB			TL_task1_yolo_reusable_SHIRTS.zip	2.39 MB

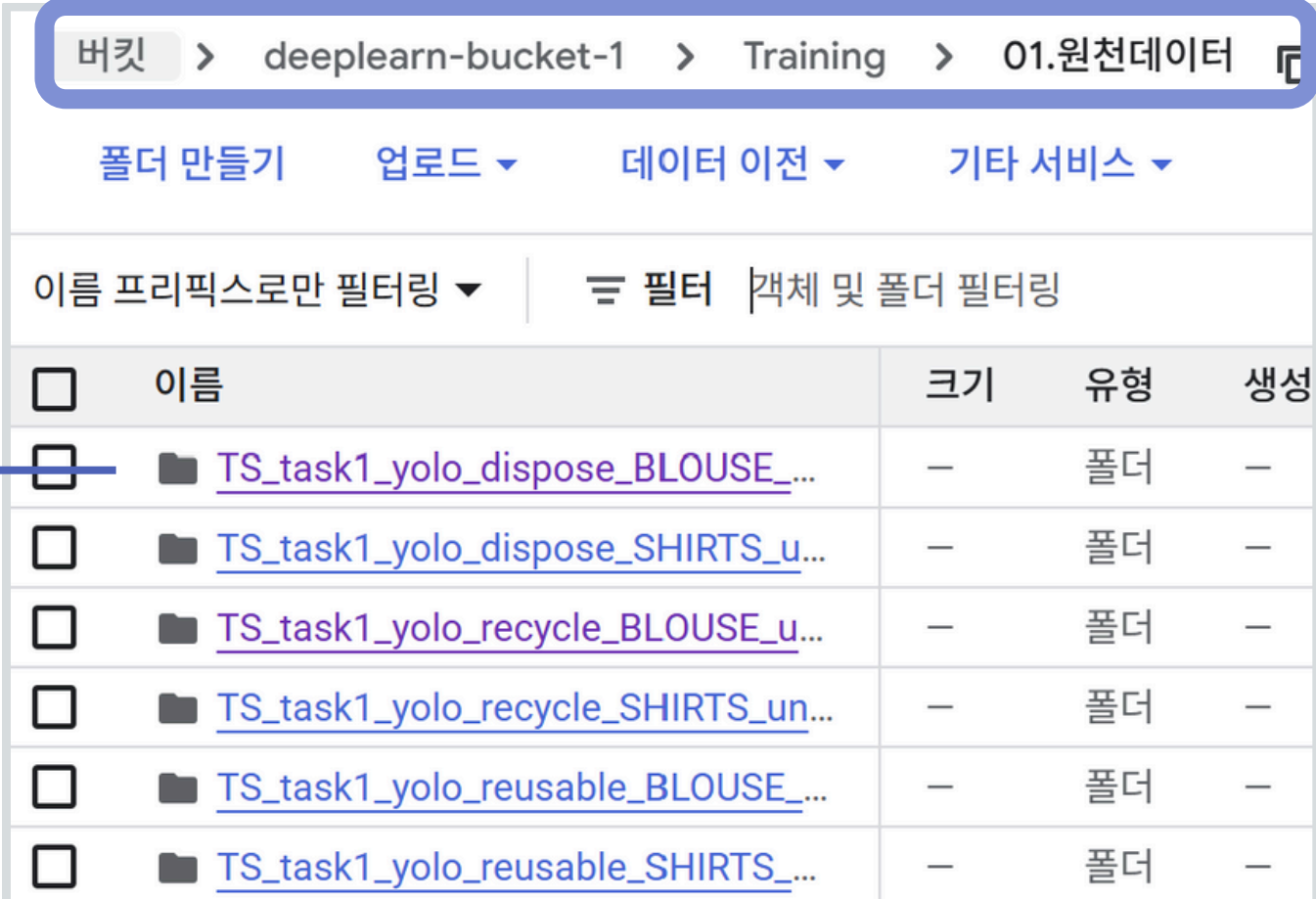
3-2 Data Storing

GCS bucket structure

gs://deeplearn-bucket-1/

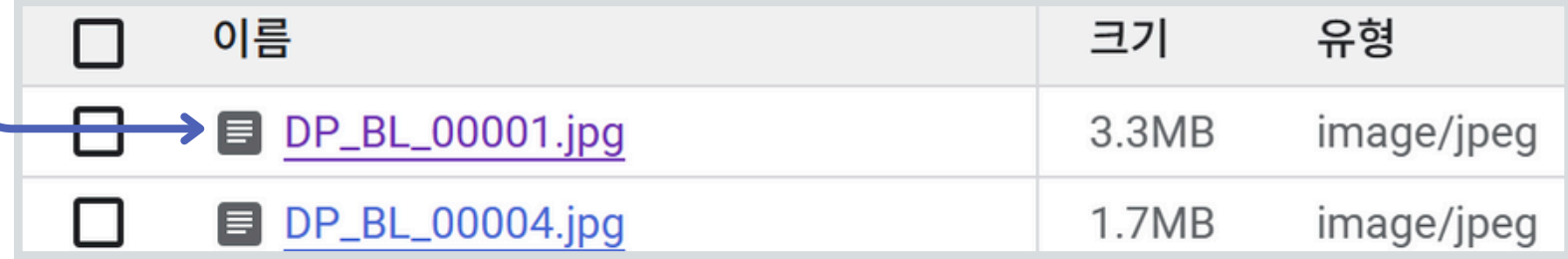
```
├── Training/
│   ├── 원천데이터/
│   │   ├── dispose/
│   │   │   ├── BLOUSE/
│   │   │   └── SHIRTS/
│   │   ├── recycle/
│   │   │   ├── BLOUSE/
│   │   │   └── SHIRTS/
│   │   └── reusable/
│   │       ├── BLOUSE/
│   │       └── SHIRTS/
│   └── 라벨데이터/
```

(files exist with the same structure)



이름	크기	유형	생성
TS_task1_yolo_dispose_BLOUSE_...	—	폴더	—
TS_task1_yolo_dispose_SHIRTS_u...	—	폴더	—
TS_task1_yolo_recycle_BLOUSE_u...	—	폴더	—
TS_task1_yolo_recycle_SHIRTS_un...	—	폴더	—
TS_task1_yolo_reusable_BLOUSE_...	—	폴더	—
TS_task1_yolo_reusable_SHIRTS_...	—	폴더	—

inside each folder



이름	크기	유형
DP_BL_00001.jpg	3.3MB	image/jpeg
DP_BL_00004.jpg	1.7MB	image/jpeg

→ enabling a stable structure, fast access during₁₅
model training

3-2 Data Storing : Challenge

Number of image data(.png)

```
[1] Training/01.원천데이터/TS_task1_yolo_dispose_BLOUSE_unzipped/  
총 이미지 개수: 438 (jpg: 438, png: 0)  
  
[2] Training/01.원천데이터/TS_task1_yolo_dispose_SHIRTS_unzipped/  
총 이미지 개수: 520 (jpg: 520, png: 0)  
  
[3] Training/01.원천데이터/TS_task1_yolo_recycle_BLOUSE_unzipped/  
총 이미지 개수: 1184 (jpg: 1184, png: 0)  
  
[4] Training/01.원천데이터/TS_task1_yolo_recycle_SHIRTS_unzipped/  
총 이미지 개수: 1198 (jpg: 1198, png: 0)  
  
[5] Training/01.원천데이터/TS_task1_yolo_reusable_BLOUSE_unzipped/  
총 이미지 개수: 2074 (jpg: 2074, png: 0)  
  
[6] Training/01.원천데이터/TS_task1_yolo_reusable_SHIRTS_unzipped/  
총 이미지 개수: 4142 (jpg: 4142, png: 0)
```

Number of labeled data (.json)

```
[1] Training/02.라벨링데이터/TL_task1_yolo_dispose_BLOUSE_unzipped/  
총 JSON 파일 개수: 438  
  
[2] Training/02.라벨링데이터/TL_task1_yolo_dispose_SHIRTS_unzipped/  
총 JSON 파일 개수: 520  
  
[3] Training/02.라벨링데이터/TL_task1_yolo_recycle_BLOUSE_unzipped/  
총 JSON 파일 개수: 1184  
  
[4] Training/02.라벨링데이터/TL_task1_yolo_recycle_SHIRTS_unzipped/  
총 JSON 파일 개수: 1198  
  
[5] Training/02.라벨링데이터/TL_task1_yolo_reusable_BLOUSE_unzipped/  
총 JSON 파일 개수: 2074  
  
[6] Training/02.라벨링데이터/TL_task1_yolo_reusable_SHIRTS_unzipped/  
총 JSON 파일 개수: 4142
```

→ Total 9,556 items, correspondence
between images and labels

04 Object Detection:

YOLOv8

04 Object Detection: YOLOv8

YOLOv8 Overview

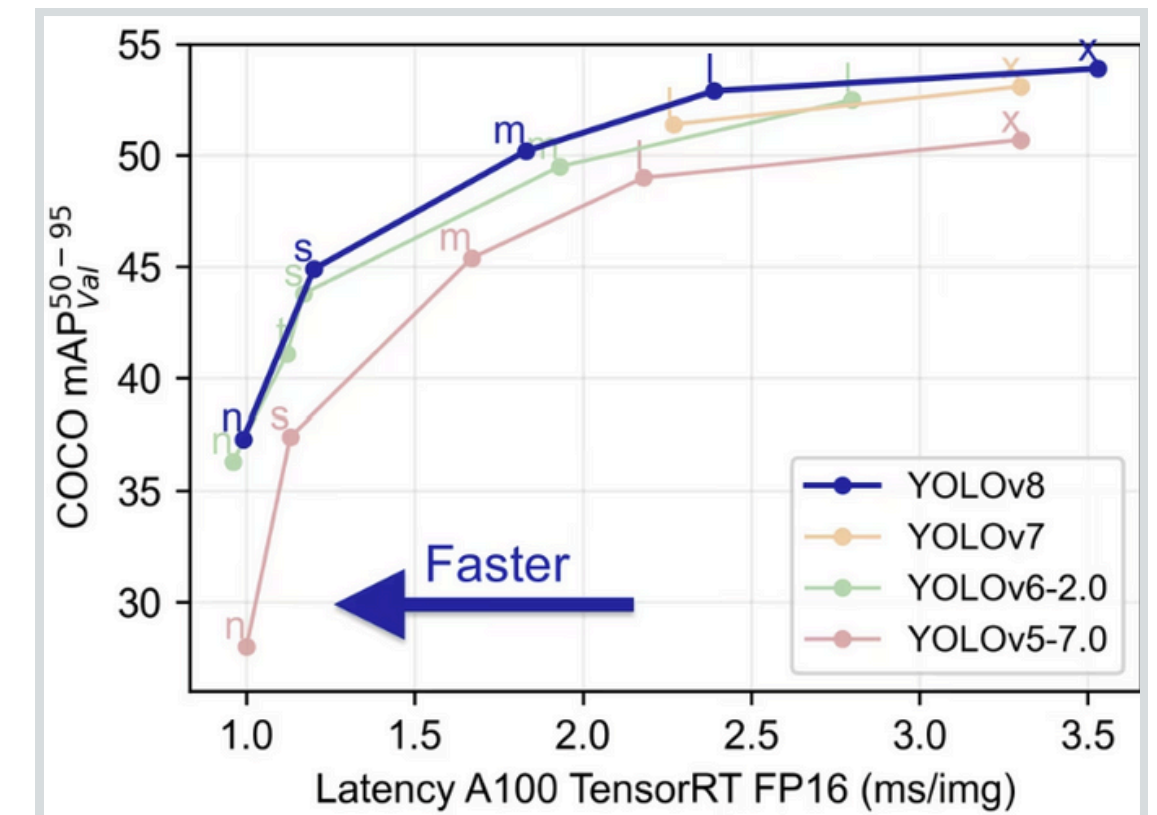
- Object detection model developed by Ultralytics
- Deep learning-based speed-accuracy optimized model
- streamlined design suitable for diverse applications

→ In this project,

based on datasets comprised of “images containing objects + bounding boxes”

YOLOv8n model fine-tuned and utilized for clothing type and contamination detection purposes

→ Class classification according to detected contamination types (reuse / recycle / dispose)



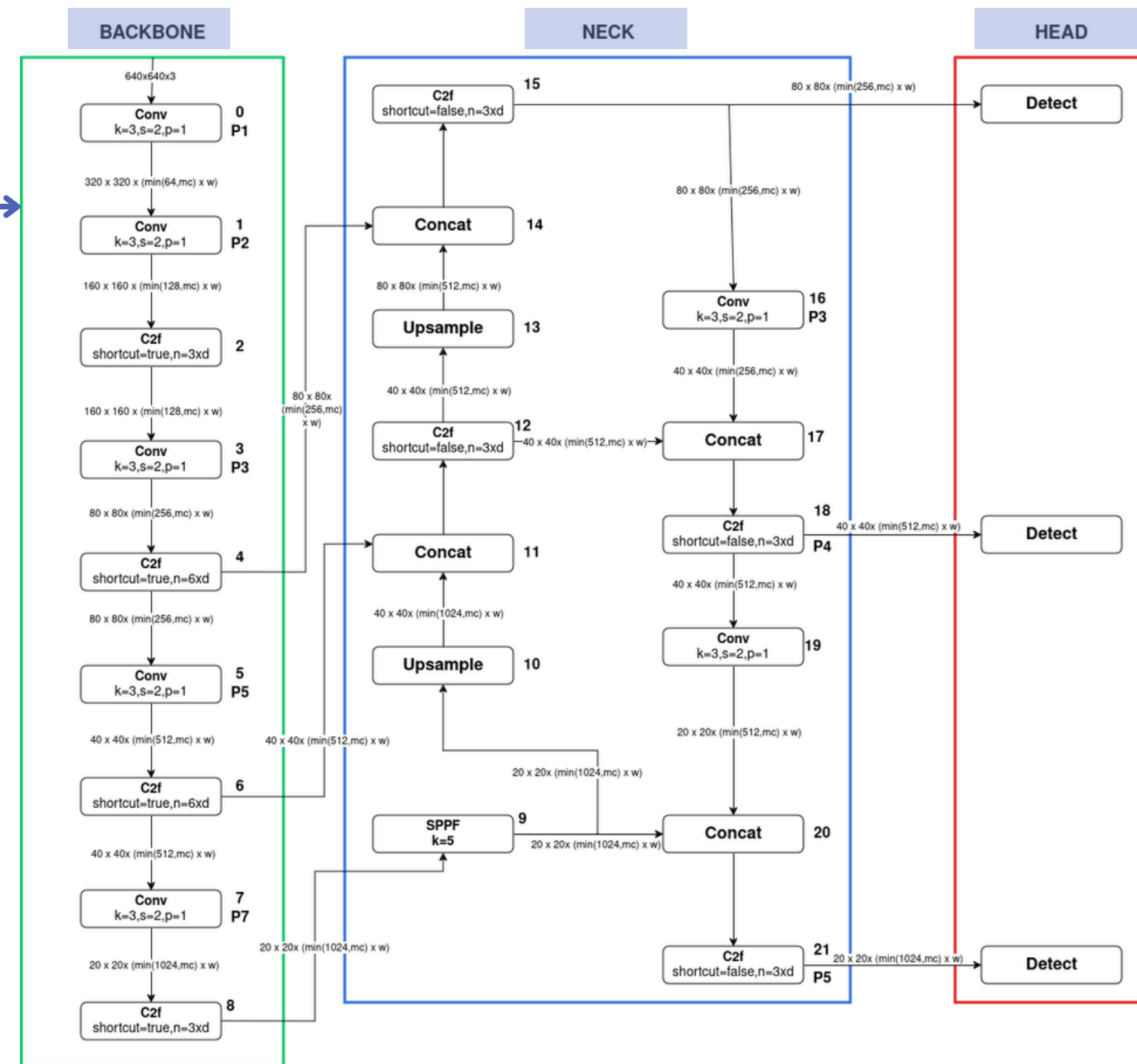
fast and accurate YOLOv8 model

04 Object Detection: YOLOv8

YOLOv8 Architecture

Backbone + Neck + Head structure

- **Backbone:** Feature extraction (CSPDarknet)
- **Neck:** Feature integration (FPN+PAN)
- **Head:** Class and location prediction



YOLOv8 architecture

04 YOLOv8: Before Training

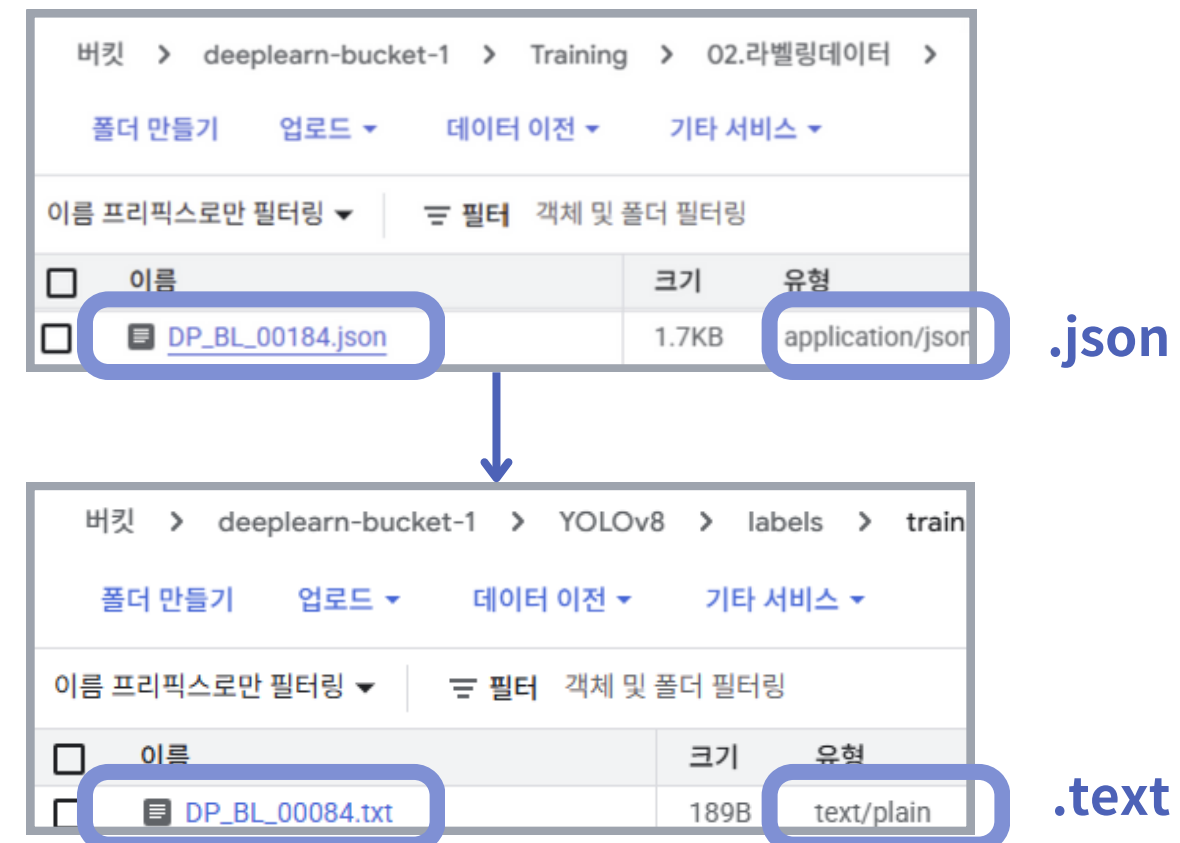
1. Data Collection

- Download images + JSON labels from GCS
- File mapping : Verify image <-> label correspondence

2. .JSON → .txt Conversion

- YOLO format: class_id x_center y_center width height (normalized)
- Coordinate normalization : Convert to 0~1 range

```
def convert_label(json_blob_path, save_txt_path, class_id=0):  
    with fs.open(json_blob_path, 'r') as f:  
        data = json.load(f)  
  
    annotations = data.get("annotations", [])  
    if not annotations:  
        return  
  
    txt_lines = []  
  
    for ann in annotations:  
        bbox = ann.get("bbox", [])  
        if len(bbox) == 4:  
            x, y, w, h = bbox  
            x_center = x + w / 2  
            y_center = y + h / 2  
            txt_line = f"{class_id} {x_center:.6f} {y_center:.6f} {w:.6f} {h:.6f}"  
            txt_lines.append(txt_line)  
  
    with open(save_txt_path, "w") as out:  
        out.write("\n".join(txt_lines))
```



04 YOLOv8: Before Training

3. Class Mapping

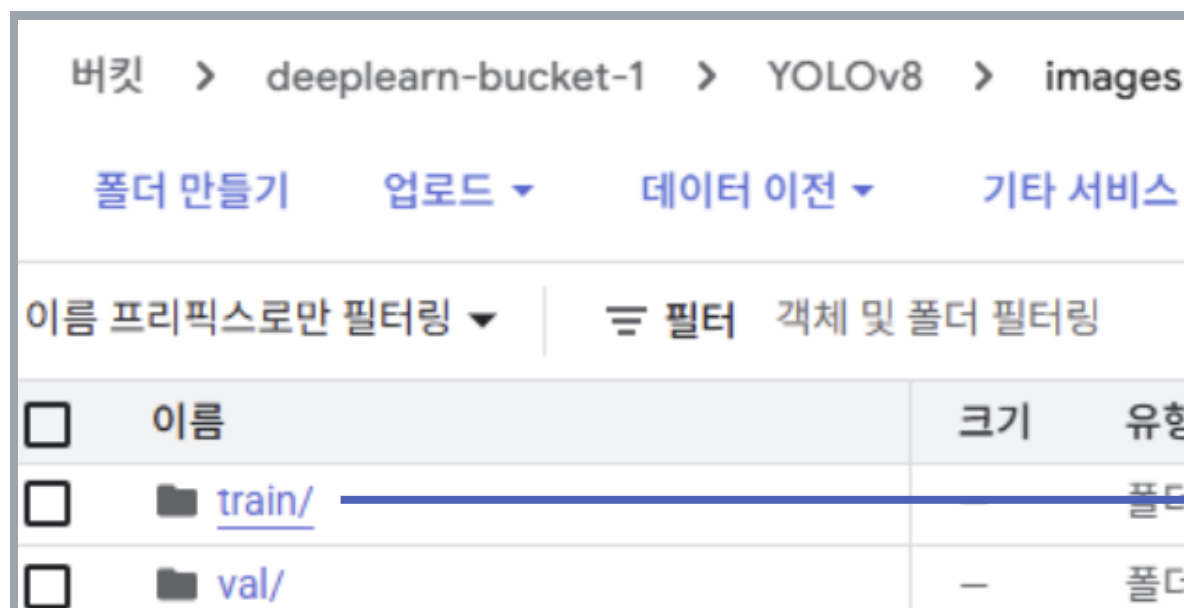
- Assign numeric IDs to 6 classes :
(dispose/recycle/reusable * blouse/shirts)

4. Data Split

- Train 80%, Validation 20%

→ Generate data.yaml : data path + class information

```
from sklearn.model_selection import train_test_split
train_imgs, val_imgs = train_test_split(image_paths, test_size=0.2, random_state=42)
```



```
# 카테고리 목록
categories = [
    ("dispose_BLOUSE", 0),
    ("dispose_SHIRTS", 1),
    ("recycle_BLOUSE", 2),
    ("recycle_SHIRTS", 3),
    ("reusable_BLOUSE", 4),
    ("reusable_SHIRTS", 5)
]
```

data.yaml: Configure train/val paths

```
path: /content/drive/MyDrive/Colab Notebooks/yolo_dataset
train: images/train
val: images/val

names:
  0: dispose_BLOUSE
  1: dispose_SHIRTS
  2: recycle_BLOUSE
  3: recycle_SHIRTS
  4: reusable_BLOUSE
  5: reusable_SHIRTS
```

04 YOLOv8: Fine-tuning

Model Training Code

```
model = YOLO("yolov8n.pt") # 경량 모델

model.train(
    data=yaml_path,
    epochs=10, # 학습 반복 횟수
    imgsz=320, # 입력 이미지 크기 (320x320)
    batch=64, # 한 번에 처리할 이미지 수
    workers=4,
    device=0,
    project=os.path.join(root_dir, "runs"),
    name="yolov8_train",
    exist_ok=True,
    save_period=1,
    amp=True,
    pretrained=True,
    patience=10, # early stopping 조건 (10 epoch 동안 성능 개선 없으면 중단)
    cache=True
)
```

```
train: Caching images (1.6GB RAM): 100%|██████████| 7628/7628
```

1st Training Result (Validation)

```
metrics = model.val() # validation set에서 평가
print(metrics.box.map) # box mAP 출력
```

```
val: Caching images (0.4GB RAM): 100%|██████████| 1907/1907 [00:33<00:00, 57.55it/s]
```

	Class	Images	Instances	Box(P	R	mAP50	mAP50-95)
	all	1907	2323	0.627	0.652	0.653	0.641
	dispose_BLOUSE	90	278	0.503	0.186	0.225	0.185
	dispose_SHIRTS	104	332	0.48	0.181	0.218	0.188
	recycle_BLOUSE	241	241	0.599	0.925	0.766	0.764
	recycle_SHIRTS	240	240	0.705	0.75	0.825	0.825
	reusable_BLOUSE	416	416	0.554	0.974	0.924	0.922
	reusable SHIRTS	816	816	0.923	0.899	0.961	0.961

- Fastest training time
- recycle/reuse classes show excellent performance
- dispose class shows low Recall and mAP

→ Suggests need for improvement in image resolution and number of epochs

04 YOLOv8: Performance Improvements

파라미터	1차 시도	2차 시도	3차 시도
모델	YOLOv8n		
이미지 크기	320	416	640
배치 크기	64		
epoch	10	40	40
Patience	10	15	10

2nd Training Results (Validation)

val: Caching images (0.7GB RAM): 100% 1907/1907 [00:33<00:00, 56.25it/s]							
Class	Images	Instances	Box(P	R	mAP50	mAP50-95)	
all	1907	2323	0.749	0.713	0.74	0.717	
dispose_BLOUSE	90	278	0.571	0.311	0.369	0.3	
dispose_SHIRTS	104	332	0.632	0.328	0.37	0.301	
recycle_BLOUSE	241	241	0.754	0.9	0.882	0.88	
recycle_SHIRTS	240	240	0.798	0.854	0.892	0.892	
reusable_BLOUSE	416	416	0.823	0.923	0.954	0.954	
reusable_SHIRTS	816	816	0.913	0.96	0.975	0.975	

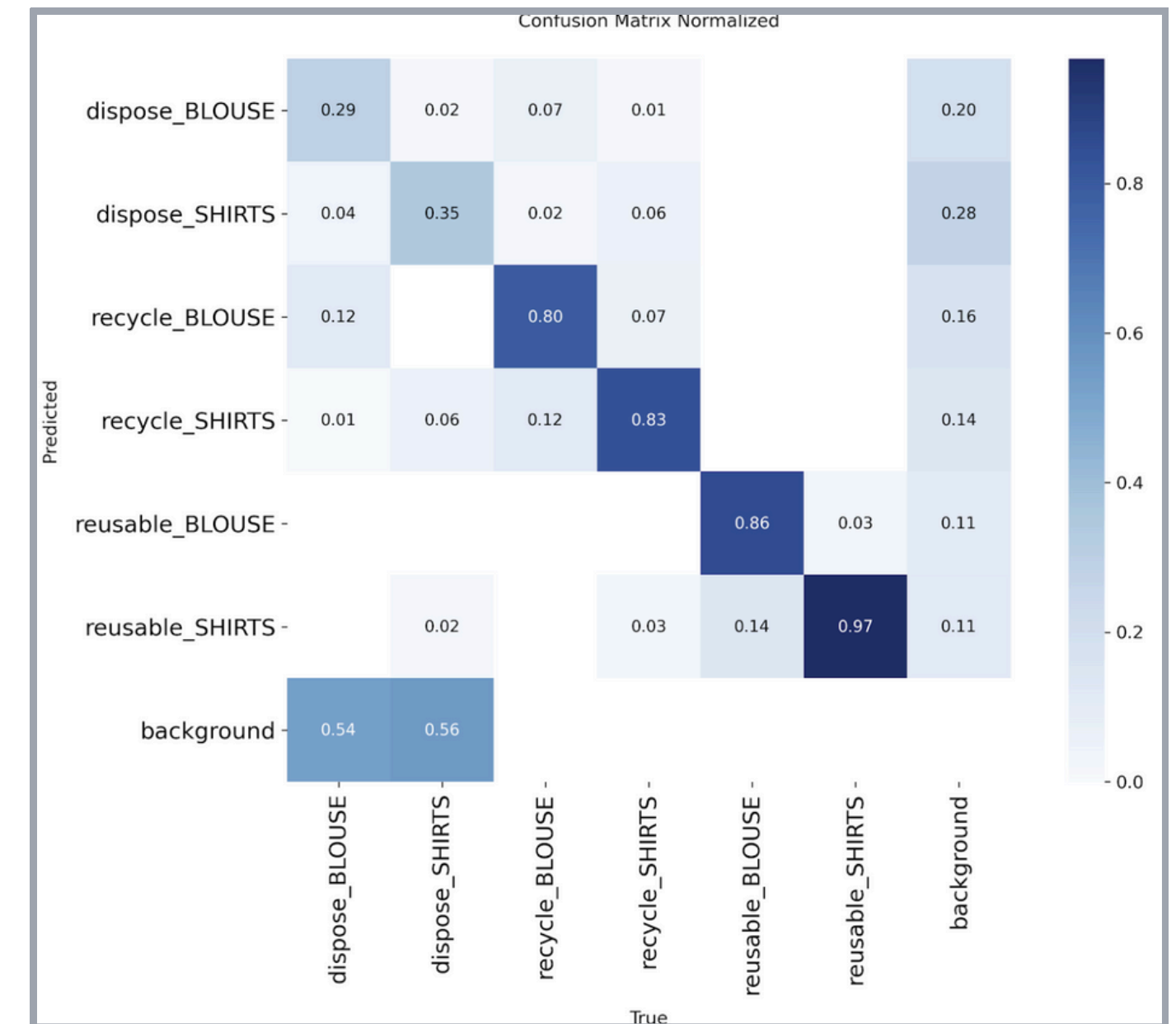
- Overall performance improvement confirmed compared to 1st training
- recycle/reuse classes show excellent performance
- dispose class shows improvement, but still has low Recall and mAP performance

04 YOLOv8: Final Model

Final Model Results (Validation)

val: Caching images (1.6GB RAM): 100% 1907/1907 [00:35<00:00, 53.10it/s]							
Class	Images	Instances	Box(P	R	mAP50	mAP50-95)	
all	1907	2323	0.752	0.718	0.752	0.723	
dispose_BLOUSE	90	278	0.623	0.32	0.401	0.301	
dispose_SHIRTS	104	332	0.565	0.337	0.385	0.314	
recycle_BLOUSE	241	241	0.755	0.888	0.889	0.887	
recycle_SHIRTS	240	240	0.755	0.887	0.904	0.904	
reusable_BLOUSE	416	416	0.906	0.883	0.964	0.964	
reusable SHIRTS	816	816	0.906	0.99	0.967	0.967	

- Improved precision and mAP compared to 2nd training
 - recycle/reuse classes maintain precision above 0.96
 - dispose class still has low Recall and mAP, but shows significant improvement compared to previous
- By applying resolution 640, micro-object detection capability improved



Normalized Confusion Matrix

04 YOLOv8: Predicted Results

train_batch.jpg

- Training image batch visualization
- Images used during training and GT bounding boxes drawn on them
- Verify labeling quality is normal



```
# 카테고리 목록
categories = [
    ("dispose_BLOUSE", 0),
    ("dispose_SHIRTS", 1),
    ("recycle_BLOUSE", 2),
    ("recycle_SHIRTS", 3),
    ("reusable_BLOUSE", 4),
    ("reusable_SHIRTS", 5)
]
```

Confirm dispose_SHIRTS is correctly labeled

val_batch_pred.jpg

- Validation batch prediction visualization
 - model predictions on validation dataset
- predicting bounding boxes + class + confidence score
- enables identification difficult to see with mAP numbers



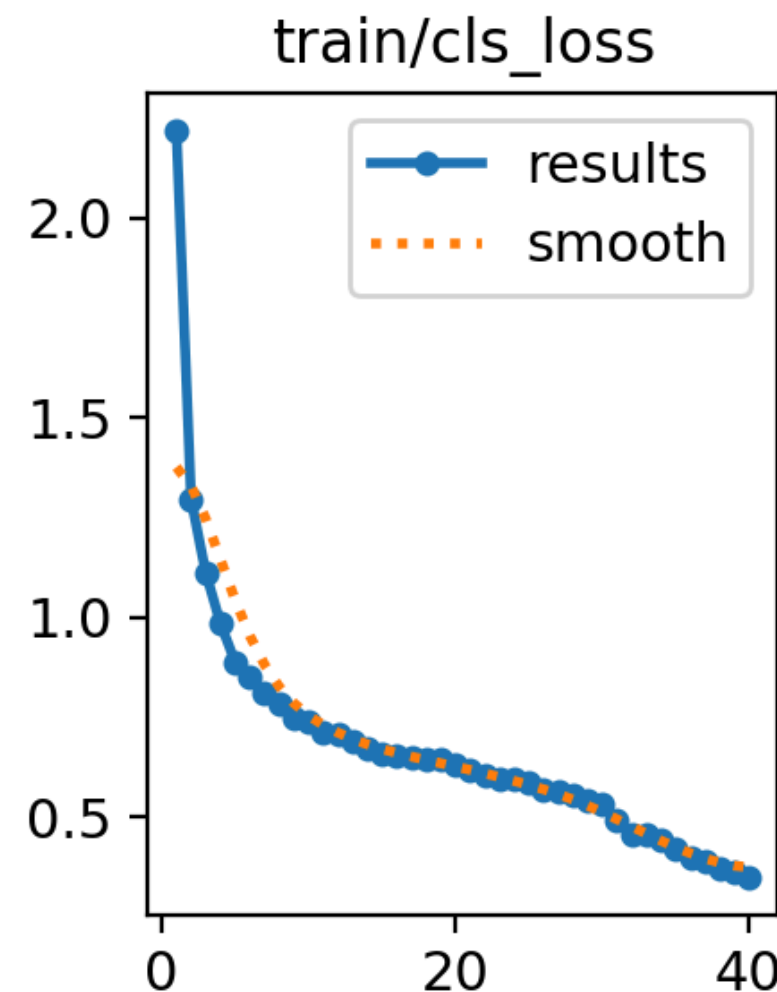
pred (예측)

labels (정답)

For patterns that are complex or decorative clothing,
visual boundaries of contamination become blurred,
reducing detection accuracy

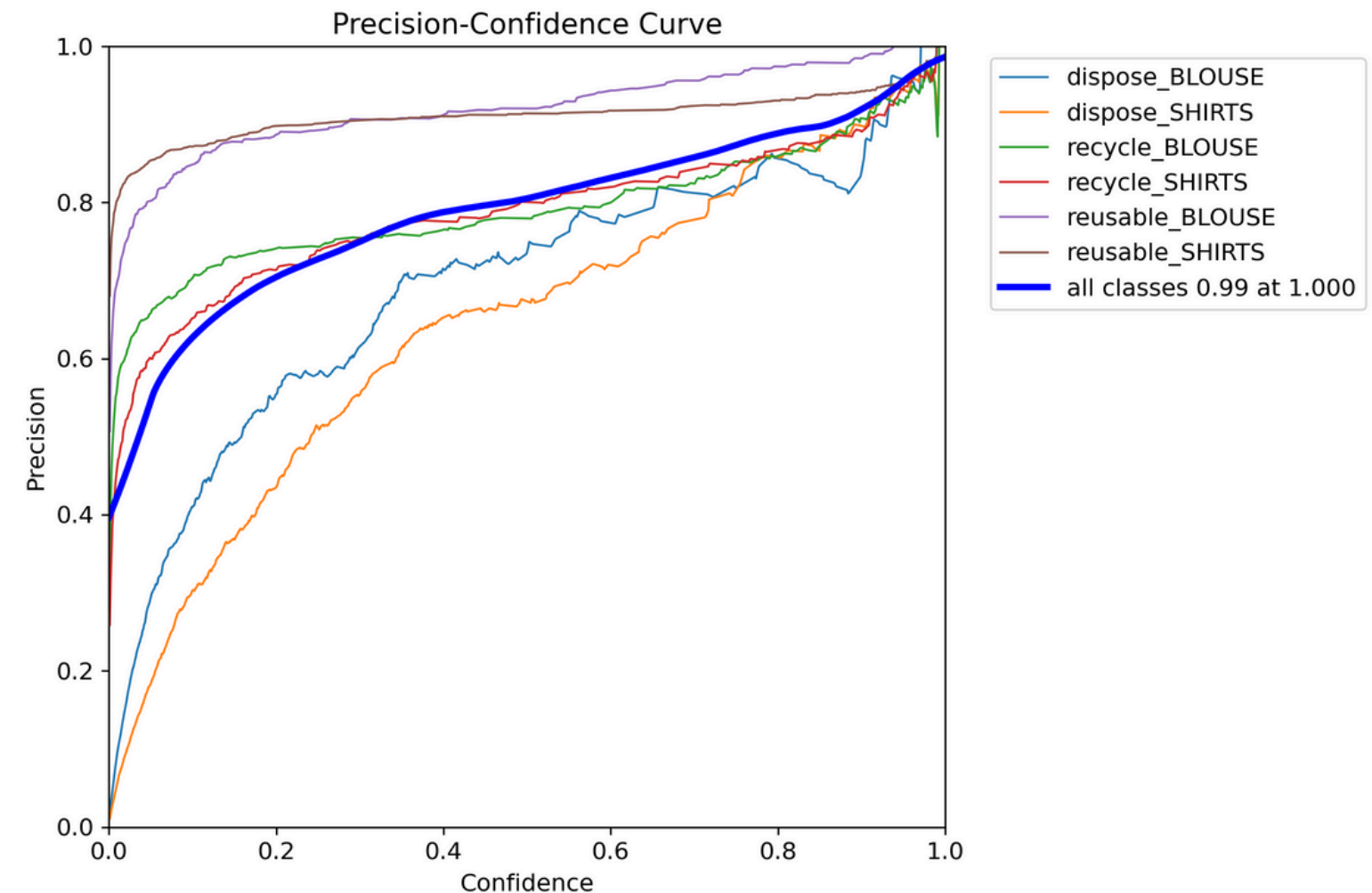
04 YOLOv8: Performance

train/cls_loss



- cls_loss continues to decrease
→ model is progressively learning accurate class predictions

Precision-Confidence Curve



- curve excluding dispose is generally maintained at high levels
→ predictions with high confidence are actually accurate²⁶

05 Image Captioning

: BLIP

05 Image Captioning : BLIP

BLIP : Bootstrapping Language-Image Pretraining

- Multimodal vision model proposed by Salesforce Research
- Strong performance in simultaneous understanding and generation of text-images
- Specialized for image captioning, VQA (Visual Question Answering), and text-based image retrieval

In this project,

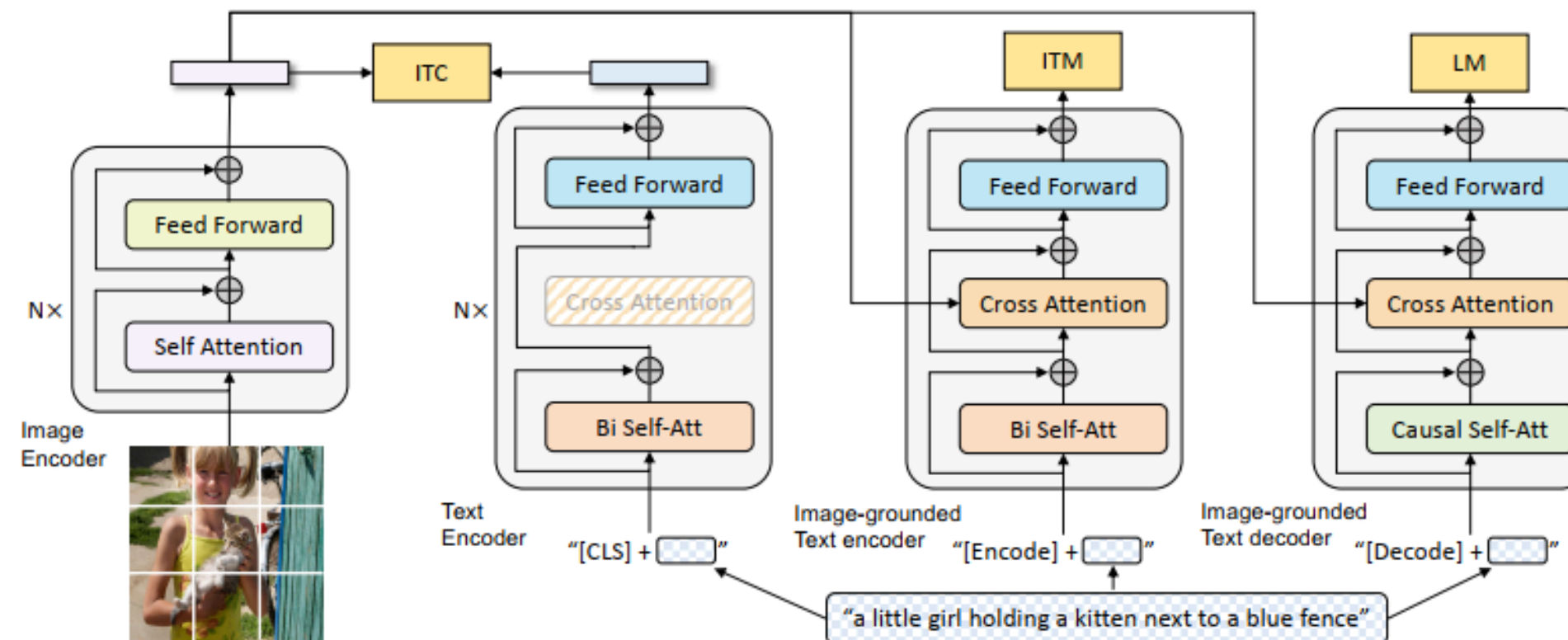
Dataset comprised of “image + caption” → **Training** → **Generate appropriate captions** when new images are input

BLIP model fine-tuned for Korean language and clothing domain.

05 Image Captioning : BLIP

BLIP Structure

- Image Encoder + Text Encoder / Decoder
- Image and text data encoded separately using ViT and BERT respectively, transformer-based text decoder



05 BLIP : Before Training

1. Load from GCS and convert to JSONL file

```
# ----- 전처리 및 저장 루프 -----
for prefix in prefixes:
    dir_name = prefix.split("/")[-1]
    local_dir = os.path.join(local_base, dir_name)
    os.makedirs(local_dir, exist_ok=True)

    # 1. GCS → Colab 다운로드
    gcs_full_path = f"gs://{bucket}/{prefix}/*.json"
    print(f"📄 Downloading from {gcs_full_path} ...")
    !gsutil -m cp $gcs_full_path $local_dir/

    # 2. JSON → JSONL로 변환
    jsonl_lines = []
    json_files = sorted(glob(os.path.join(local_dir, "*.json")))
    for path in json_files:
        try:
            with open(path, "r", encoding="utf-8") as f:
                data = json.load(f)
                file_name = data.get("meta_information", {}).get("file_name")
                caption = data.get("image_caption", {}).get("caption")
                if file_name and caption:
                    jsonl_lines.append(json.dumps({"image": file_name, "caption": caption}, ensure_ascii=False))
        except Exception as e:
            print(f"⚠️ {os.path.basename(path)} 처리 실패: {e}")

    # 3. 로컬에 jsonl 저장
    jsonl_filename = f"{dir_name}.jsonl"
    jsonl_path = os.path.join(local_base, jsonl_filename)
    with open(jsonl_path, "w", encoding="utf-8") as f:
        f.write("\n".join(jsonl_lines))
    print(f"✅ {jsonl_filename} 저장 완료 ({len(jsonl_lines)}개 항목)")
```

All .json files in each prefix are loaded
Extract “file_name” and “caption” information
Gather the information into a single .jsonl file

버킷 > deeplearn-bucket-1 > BLIP > jsonl

폴더 만들기 업로드 데이터 이전 기타 서비스

이름 프리픽스로만 필터링 필터 객체 및 폴더 필터링

☐	이름	크기	유형	생성일 ?
☐	TL_task1_yolo_dispose_BLOUSE_u...	79.4KB	application/octet-stream	2025. 6. 19. PM 2:25:25
☐	TL_task1_yolo_dispose_SHIRTS_u...	101KB	application/octet-stream	2025. 6. 19. PM 2:25:35
☐	TL_task1_yolo_recycle_BLOUSE_u...	214.9KB	application/octet-stream	2025. 6. 19. PM 2:25:52
☐	TL_task1_yolo_recycle_SHIRTS_un...	239.8KB	application/octet-stream	2025. 6. 19. PM 2:26:08
☐	TL_task1_yolo_reusable_BLOUSE_...	334.2KB	application/octet-stream	2025. 6. 19. PM 2:26:34
☐	TL_task1_yolo_reusable_SHIRTS_u...	682.7KB	application/octet-stream	2025. 6. 19. PM 2:27:20

.jsonl files stored in GCS

.jsonl files for each prefix

dispose_BLOUSE	dispose_SHIRTS
recycle_BLOUSE	recycle_SHIRTS
reuse_BLOUSE	reuse_SHIRTS

05 BLIP : Preprocessing

2. Image-text data preprocessing for BLIP training

```
# 전처리 함수 정의
def transform(example):
    image_path = os.path.join(image_dir, example["image"])
    try:
        image = Image.open(image_path).convert("RGB")
    except:
        image = Image.new("RGB", (224, 224))
    inputs = processor(images=image, text=example["caption"], return_tensors="pt", padding="max_length")
    return {
        "input_ids": inputs.input_ids[0],
        "attention_mask": inputs.attention_mask[0],
        "pixel_values": inputs.pixel_values[0],
        "labels": inputs.input_ids[0]
    }

# 전처리 적용
dataset = dataset.map(transform, remove_columns=["image", "caption"])

dataset.save_to_disk("/content/preprocessed_blip_dataset")
print("📁 전처리된 데이터셋 저장 완료")
```

전체 샘플 수: 9556
/usr/local/lib/python3.11/dist-packages/datasets/table.py:1395: FutureWarning: promote has been super
block_group = [InMemoryTable(cls._concat_blocks(list(block_group), axis=axis))]
/usr/local/lib/python3.11/dist-packages/datasets/table.py:1421: FutureWarning: promote has been super
table = cls._concat_blocks(blocks, axis=0)
Map: 100%  9556/9556 [44:36<00:00, 3.42 examples/s]
Saving the dataset (34/34 shards): 100%  9556/9556 [00:48<00:00, 165.32 examples/s]
📁 전처리된 데이터셋 저장 완료

버킷 > deeplearn-bucket-1 > BLIP > preprocessed_blip_dataset 

[폴더 만들기](#) [업로드](#) [데이터 이전](#) [기타 서비스](#)

이름 프리픽스로만 필터링  필터 객체 및 폴더 필터링

☐	이름	크기	유형
☐	 data-00000-of-00034.arrow	500.8MB	application/vnd.apache.arrow.file
☐	 data-00001-of-00034.arrow	500.8MB	application/vnd.apache.arrow.file
☐	 data-00002-of-00034.arrow	499MB	application/vnd.apache.arrow.file
☐	 data-00003-of-00034.arrow	499MB	application/vnd.apache.arrow.file
☐	 data-00004-of-00034.arrow	499MB	application/vnd.apache.arrow.file
☐	 data-00005-of-00034.arrow	499MB	application/vnd.apache.arrow.file
☐	 data-00006-of-00034.arrow	499MB	application/vnd.apache.arrow.file
☐	 data-00007-of-00034.arrow	499MB	application/vnd.apache.arrow.file

Stored in GCS의 deeplearn-bucket-1/
BLIP/preprocessed_blip_dataset/

Preprocessed dataset for BLIP training

05 BLIP : Fine-tuning

3. Trainer-based model fine-tuning

1st Attempt

```
training_args = TrainingArguments(  
    output_dir="./blip_finetuned",  
    per_device_train_batch_size=8,  
    per_device_eval_batch_size=8,  
    num_train_epochs=3,  
    eval_strategy="epoch",  
    save_strategy="epoch",  
    logging_dir="./logs",  
    logging_steps=10,  
    learning_rate=5e-5,  
    remove_unused_columns=False,  
    fp16=True,  
    report_to="none",  
    load_best_model_at_end=True,  
    metric_for_best_model="eval_loss"  
)
```

7. Trainer 정의

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=train_data_loader.get_train_dataset(),  
    eval_dataset=eval_data_loader.get_eval_dataset()  
)
```

8. 학습 시작

```
trainer.train()
```

[3225/3225 2:17:01, Epoch 3/3]

Epoch	Training Loss	Validation Loss
1	0.224300	0.207488
2	0.138100	0.167237
3	0.112300	0.150097

There were missing keys in the checkpoint model loaded: ['text_decoder.cls.pr
TrainOutput(global_step=3225, training_loss=0.2910142052820487, metrics={'tra
0.392, 'total_flos': 1.531033022740562e+19, 'train_loss': 0.2910142052820487,

validation loss decreases
as epochs progress

2nd Attempt (Changing parameters)

```
training_args = TrainingArguments(  
    output_dir="./blip_finetuned_v2",  
    per_device_train_batch_size=8,  
    per_device_eval_batch_size=8,  
    num_train_epochs=5, # 증가  
    eval_strategy="epoch",  
    save_strategy="epoch",  
    logging_dir="./logs",  
    logging_steps=10,  
    learning_rate=3e-5, # 감소  
    gradient_accumulation_steps=2, # 추가  
    remove_unused_columns=False,  
    fp16=True,  
    report_to="none",  
    load_best_model_at_end=True,  
    metric_for_best_model="eval_loss"  
)
```

Trainer 정의

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_data_loader=train_data_loader,  
    eval_data_loader=eval_data_loader  
)
```

학습 시작

```
trainer.train()
```

increased num_train_epochs
decreased learning_rate
added gradient_accumulation_step

validation loss continuously
increases as epochs progress
(high likelihood of overfitting)

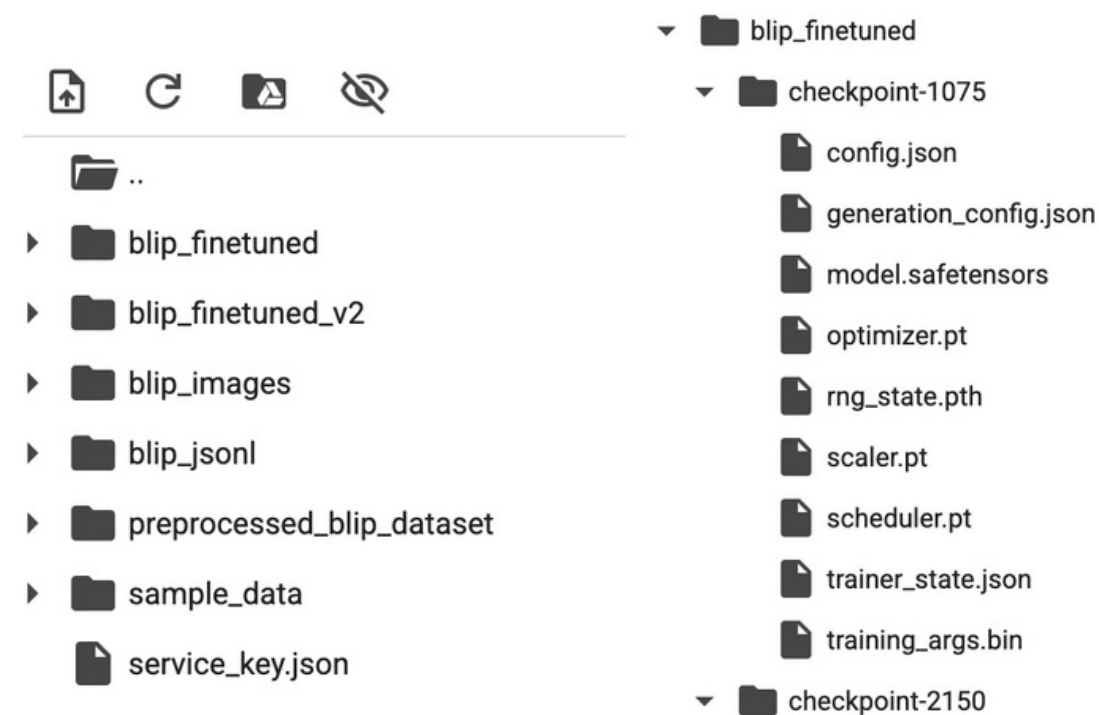
[2690/2690 3:45:26, Epoch 5/5]

Epoch	Training Loss	Validation Loss
1	0.126500	0.157299
2	0.099500	0.157422
3	0.072100	0.160317
4	0.050100	0.168785
5	0.037200	0.175350

There were missing keys in the checkpoint model loaded: ['text_decoder.cls.predictions.decoder.bias'].
TrainOutput(global_step=2690, training_loss=0.07884542105587884, metrics={'train_runtime': 1352.304,
0.199, 'total_flos': 2.551721704567603e+19, 'train_loss': 0.07884542105587884, 'epoch': 5.0})

05 BLIP : Data Storage

Directory structure



Saving training at each checkpoint
update method applies to next steps

버킷 > deeplearn-bucket-1 > BLIP

폴더 만들기 업로드 데이터 이전 기타 서비스

이름 프리픽스로만 필터링 필터 객체 및 폴더 필터링

☐	이름	크기	유형
☐	blip_finetuned/	—	폴더
☐	blip_finetuned_v2/	—	폴더
☐	jsonl/	—	폴더
☐	preprocessed_blip_dataset/	—	폴더

4. Caption generation for test data (Evaluation)

```
# test 데이터에 대한 정답 캡션 생성
import torch
from tqdm import tqdm

references = []
predictions = []

device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)
model.eval()

for sample in tqdm(eval_dataset):
    pixel_values = torch.tensor(sample["pixel_values"]).unsqueeze(0).to(device)
    with torch.no_grad():
        generated_ids = model.generate(pixel_values=pixel_values, max_new_tokens=50)
        generated_caption = processor.decode(generated_ids[0], skip_special_tokens=True)
        predictions.append(generated_caption)

# 정답 캡션 (input_ids → 텍스트 복원)
label_ids = sample["labels"]
label_text = processor.tokenizer.decode(label_ids, skip_special_tokens=True)
references.append(label_text)
```

100% | 956/956 [18:51<00:00, 1.18s/it]

GCS directory storage structure

05 BLIP : Examples

4. Caption generation for test data (Example) :

```
# 실제 캡션 가져오기
with open(jsonl_path, "r", encoding="utf-8") as f:
    for line in f:
        data = json.loads(line)
        if data["image"] == target_image:
            real_caption = data["caption"]
            break

if real_caption is None:
    raise ValueError(f"{target_image}에 대한 캡션을 찾을 수 없습니다.")

# 이미지 로딩 및 모델 입력
image_path = f"/content/blip_images/{target_image}"
image = Image.open(image_path).convert("RGB")
inputs = processor(images=image, return_tensors="pt").to(model.device)

# 캡션 생성
with torch.no_grad():
    generated_ids = model.generate(**inputs, max_length=128)
    predicted_caption = processor.decode(generated_ids[0], skip_special_tokens=True)

# 이미지 먼저 출력
plt.figure(figsize=(6, 6))
plt.imshow(image)
plt.axis("off")
plt.show()

# 예측 및 실제 캡션 출력
print("예측된 캡션:", predicted_caption)
print("실제 캡션:", real_caption)
```



예측된 캡션: 카라가 단추가 중앙이 흰색의 남녀 공용 긴팔 셔츠
실제 캡션: 카라가 있고 소매에 단추가 있으며 중앙에 외부오염이 있는 아이보리색 여성용 긴팔 블라우스



예측된 캡션: 흰색과 스트라이프 패턴이 중앙에 여성용 반팔 블라우스
실제 캡션: 우측 중앙에 외부오염이 있는 하늘색 여성용 반팔 드레스

05 Image Captioning : Performance

5. Performance Evaluation and Attempts

1. Three evaluation metrics for prediction assessment :

```
!pip install evaluate nltk --quiet
!pip install rouge_score --quiet

import evaluate
import nltk
nltk.download('punkt')

Preparing metadata (setup.py) ... done
Building wheel for rouge_score (setup.py) ... done
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
True
```

```
bleu = evaluate.load("bleu")
rouge = evaluate.load("rouge")
meteor = evaluate.load("meteor")
```

```
Downloading builder script: 100% 7.02k/7.02k

[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
```

BLEU
ROUGE-L
METEOR

BLEU : Evaluates precision at the word level

ROUGE-L : Evaluates the longest common subsequences

METEORS : Accuracy + Recall + Stemming / Synonym matching

• 1st Attempt

```
# BLEU expects list of references per prediction
bleu_result = bleu.compute(predictions=predictions, references=[[ref] for ref
rouge_result = rouge.compute(predictions=predictions, references=references)
meteor_result = meteor.compute(predictions=predictions, references=references)

print("Evaluation Metrics on Validation Set")
print(f"BLEU score: {bleu_result['bleu']:.4f}")
print(f"ROUGE-L: {rouge_result['rougeL']:.4f}")
print(f"METEOR: {meteor_result['meteor']:.4f}")
```

```
Evaluation Metrics on Validation Set
BLEU score: 0.1174
ROUGE-L: 0.0105
METEOR: 0.2810
```

BLEU: 0.1174 , ROUGE-L: 0.0105, METEOR: 0.2810

• 2nd Attempt

```
# BLEU expects list of references per prediction
bleu_result = bleu.compute(predictions=predictions, references
rouge_result = rouge.compute(predictions=predictions, referenc
meteor_result = meteor.compute(predictions=predictions, refere

print("Evaluation Metrics on Validation Set")
print(f"BLEU score: {bleu_result['bleu']:.4f}")
print(f"ROUGE-L: {rouge_result['rougeL']:.4f}")
print(f"METEOR: {meteor_result['meteor']:.4f}")
```

```
Evaluation Metrics on Validation Set
BLEU score: 0.1133
ROUGE-L: 0.0188
METEOR: 0.2779
```

BLEU: 0.1133 , ROUGE-L: 0.0188, METEOR: 0.2779

06 Applications

06 Potential and Applications

Commercial Viability Strategy - Linkage with secondhand trading platforms

What if this deep learning model is applied to a secondhand trading platform?

- **False listing detection and verification** support based on uploaded images

Example : automatic warning when severe contamination is detected on clothing images marked as **clean condition**



- Providing convenience functions for sellers

Image-based automatic description generation (Captioning) - automatic extraction of product name, color, features to automate post creation



Application : Automated clothing classification and description generation system
based on product images from Jungonara (a Korean secondhand platform)

06 Potential and Applications

Jungonara product image-based clothing classification and description generation

1. Web Crawling

- Target : Jungonara clothing category products (total 1,620 items)
- Method : Crawl pages 1 to 20 sorted by lowest price (27 items per page)
- Used packages : requests, BeautifulSoup

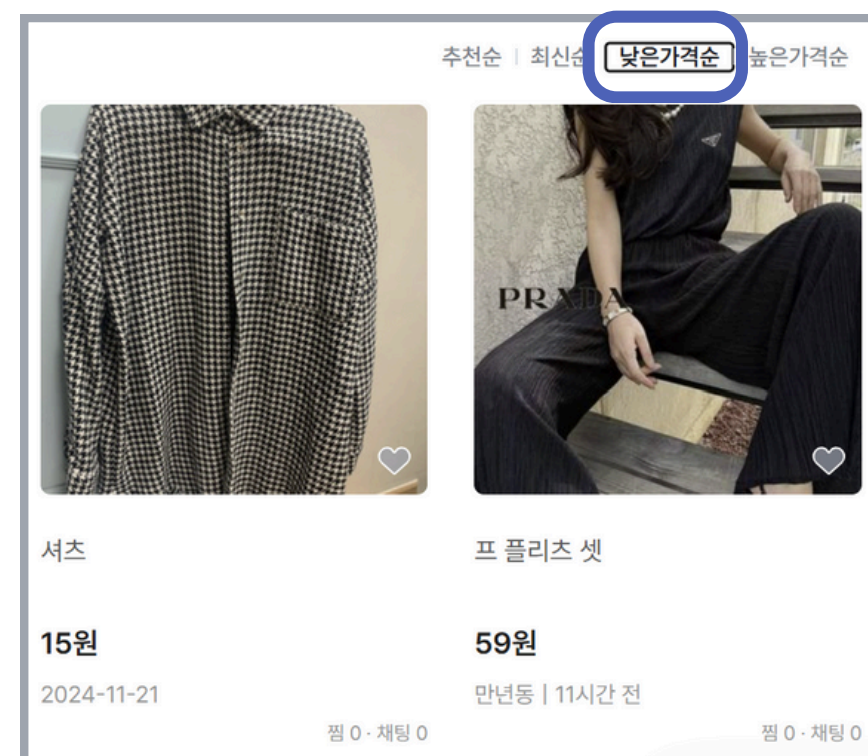
→ Crawling pipeline for service construction based on real usage data

```
url = f"https://web.joongna.com/search?category=1053&sort=PRICE_ASC_SORT&page={page}"
response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.text, 'html.parser')

for li in soup.find_all('li'):
    a_tag = li.find('a', href=True)
    if a_tag and a_tag['href'].startswith('/product/'):
        product_url = "https://web.joongna.com" + a_tag['href']

        img_tag = a_tag.find('img')
        img_url = img_tag['src'] if img_tag and img_tag.has_attr('src') else None

        title_tag = a_tag.find('h2')
        title = title_tag.text.strip() if title_tag else None
```



Partial crawling code

06 Potential and Applications

Collected example data

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1620 entries, 0 to 1619
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   title       1600 non-null   object
1   price       1600 non-null   object
2   product_url 1620 non-null   object
3   img_url     1600 non-null   object
4   wish_count  1600 non-null   object
5   chat_count  1600 non-null   object
dtypes: object(6)
memory usage: 76.1+ KB
```

	title	price	image_tag	wish_count	chat_count	price_num
0	자라 크롭탑 오프숄더 블라우스 구매요	1원		0	0	1.0
1	타미힐피거 남성용 슬림핏s 화이트 셔츠	1원		0	0	1.0
⋮						
1599	여성 긴팔 블라우스 새상품	8,000원		0	0	8000.0

Product name text analysis

```
words = df['title'].dropna().apply(lambda x: re.findall(r'\bWw+Wb', x.lower()))
flat_words = [word for sublist in words for word in sublist]
Counter(flat_words).most_common(40)

[('블라우스', 730),
 ('셔츠', 274),
 ('여성', 185),
 ('새상품', 105),
 ('66', 101),
 ('반팔', 97),
 ('55', 90),
 ('남방', 85),
 ('77', 56),
 ('티셔츠', 55),
 ('m', 52),
 ('체크', 50),
 ('여름', 48),
 ('쉬폰', 47),
 ('시스루', 45),
 ('레이스', 45),
 ('화이트', 44),
 ('free', 44),
 ('새것', 42),
 ('크롭', 41),
 ⋮]
```

→ Generally, titles include clothing type or size

06 LiClo Service Development

Gradio-based Automated System

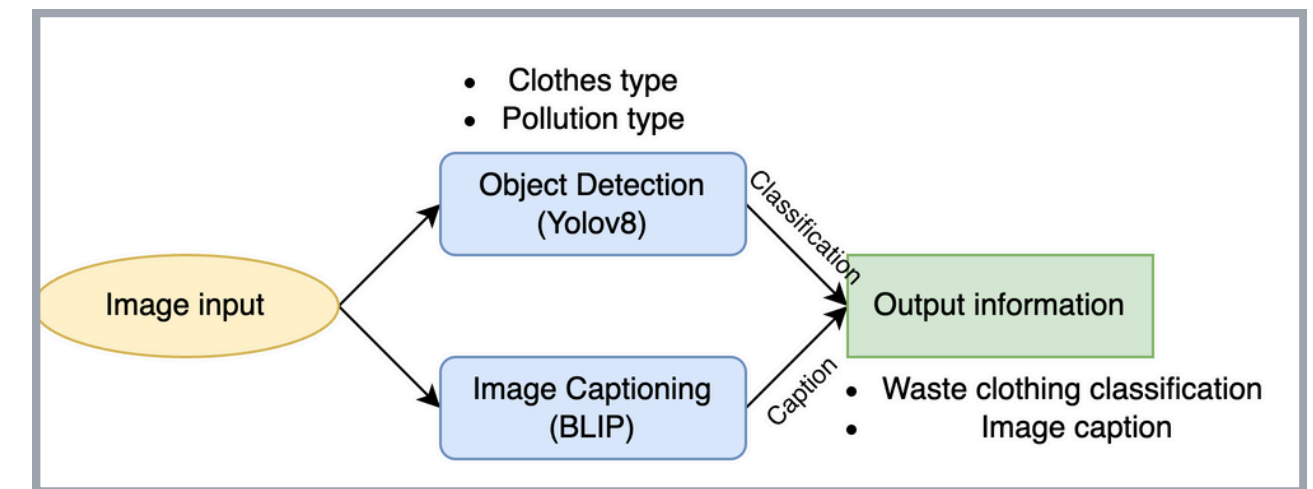
- LiClo (Recycle + Clothing): Automated service built by our team

Key Components

- YOLOv8: Clothing shape and contamination object detection + classification (dispose/recycle/reuse)
- BLIP: Image caption generation (automatic clothing description)
- Gradio: Real-time prediction results provided upon user image upload

Prediction results

- Object Detection Bounding Box visualization
- Detected class name (one of dispose/recycle/reuse)
- Confidence score for each class
- BLIP-based natural language image description generation



→ Where users upload images,
detection + classification + caption generation
are automatically performed.

06 LiClo Service Demo

Gradio-based Automated System

LiClo 서비스

Recycle + Clothing 시스템
YOLOv8로 객체 및 오염 부분 감지를 통한 dispose/recycle/reuse 분류 + BLIP 이미지 캡션 생성까지 자동으로 실행됩니다.



Clear

Submit

dispose / reuse / recycle 확률

Dispose: 0.0%
Recycle: 0.0%
Reuse: 90.7%

이미지 캡션

이미지 캡션:
넥으로 장식이 흰색 여성용 반팔 블라우스

클래스별 감지 결과

reusable_BLOUSE (0.91)



Demonstration Video

06 Conclusion

1

Attempted to resolve environmental issues by highlighting the feasibility of recycling waste materials.

2

Built a deep learning pipeline that integrates YOLO's object detection with BLIP's natural language generation capabilities.

3

Proposed the potential for commercialization, not limited only to the public sector.

THANK YOU

감사합니다

Minji Kim, Kaeun Han
Ewha Womans University